

部署与运维手册

1. 本地一键启动

Windows 推荐方式

Start-RepoHealth.bat

启动后

- 后端 <http://127.0.0.1:8002>
- 前端 <http://127.0.0.1:5174>
- 健康检查 <http://127.0.0.1:8002/health>

2. 手动启动

后端

```
pip install -r backend/requirements.txt
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8002 --reload
```

前端

```
cd frontend
npm install
npm run dev -- --host 127.0.0.1 --port 5174
```

3. 环境变量

变量	说明
SESSION_SECRET	Session 签名密钥 生产环境必填
CORS_ORIGINS	允许访问后端的前端来源
FRONTEND_URL	OAuth 登录后跳转地址
DEEPSEEK_API_KEY	可选 LLM 诊断
OPENAI_API_KEY	可选 LLM 诊断备用
GIT_HTTP_PROXY	可选 Git 克隆代理
TRUST_PROXY_HEADERS	是否信任反向代理头
ALLOW_NPX_INSTALL	是否允许自动安装 npx 工具

4. GitHub Actions

GitHub CI 建议包含

1. Python lint
2. 后端 pytest
3. E2E 自测
4. 前端 lint
5. 前端 Vitest
6. 前端 build

5. GitLink DevOps

推荐图形流水线

GitLink trigger -> 开始 -> backend shell -> frontend shell -> 结束

backend shell

```
sh devops/gitlink/backend-ci.sh
```

frontend shell

```
sh devops/gitlink/frontend-ci.sh
```

6. 发布流程

1. 确认工作区干净
2. 执行测试和构建
3. 提交代码
4. 创建版本标签
5. 推送到 GitHub
6. 创建 GitHub Release

示例

```
git commit -m "release: stabilize v2.1.0"
git tag -a v2.1.0 -m "v2.1.0 stable UI and UX release"
git push origin master
git push origin v2.1.0
```

7. 常见故障

问题	原因	解决
前端无法连接后端	端口/CORS 不一致	确认前端 5174 后端 8002
浏览器显示旧 UI	旧 Vite 服务占用端口	关闭旧终端 重新启动
克隆 GitHub 失败	网络或代理问题	配置 <code>GIT_HTTP_PROXY</code>
分析超时	仓库过大	使用缓存或分析较小仓库
PDF 无法导出	浏览器拦截弹窗	允许弹窗后重试

8. 运维检查

发布前检查

```
python -m pytest backend/tests -q
cd frontend
npm run lint
npm test -- --run
npm run build
```